

# Problem Set 1: Solutions

## Compilers & Computational Theory (Lecture 1)

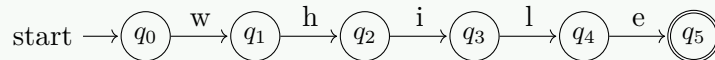
### Compiler Construction Module

## 1 Lexical analysis: building a DFA

### Problem: 1(a): a DFA for the keyword `while`

#### Solution

One state per character read so far, plus an implicit error state for any other input.



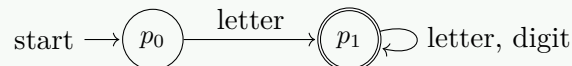
Formally  $M = (Q, \Sigma, \delta, q_0, F)$  with

$$\begin{aligned} Q &= \{q_0, q_1, q_2, q_3, q_4, q_5, q_{\text{err}}\}, \\ \Sigma &= \text{the character set (at least } \{\mathbf{w, h, i, l, e}\}), \\ \delta(q_0, \mathbf{w}) &= q_1, \delta(q_1, \mathbf{h}) = q_2, \delta(q_2, \mathbf{i}) = q_3, \delta(q_3, \mathbf{l}) = q_4, \delta(q_4, \mathbf{e}) = q_5, \\ \delta(q, x) &= q_{\text{err}} \text{ for every other pair, and } \delta(q_{\text{err}}, x) = q_{\text{err}}, \\ F &= \{q_5\}. \end{aligned}$$

$L(M) = \{\mathbf{while}\}$ : the only path to  $q_5$  spells `while`, and any deviation falls into the absorbing error state.

### Problem: 1(b): a DFA and a regular expression for identifiers

#### Solution



$Q = \{p_0, p_1, p_{\text{err}}\}$ ,  $\delta(p_0, \text{letter}) = p_1$ ,  $\delta(p_1, \text{letter}) = \delta(p_1, \text{digit}) = p_1$ , everything else to  $p_{\text{err}}$ ,  $F = \{p_1\}$ . Regular expression:

$$\text{letter} (\text{letter} \mid \text{digit})^* \quad \text{i.e.} \quad [\mathbf{a-zA-Z}][\mathbf{a-zA-Z0-9}]^*.$$

The first symbol must be a letter (so `9x` is not an identifier); afterwards any mix, including none.

### Problem: 1(c): combining the two in one lexer

#### Solution

Regular languages are closed under union, so  $L_{\text{while}} \cup L_{\text{id}}$  is regular and has a single DFA. Concretely: add a new start state with  $\epsilon$ -edges into  $q_0$  and  $p_0$  (an NFA), then run subset construction. The resulting DFA tracks *both* machines at once: after reading **w** it is in the set  $\{q_1, p_1\}$ , after **wh** in  $\{q_2, p_1\}$ , and after **while** in  $\{q_5, p_1\}$ , a state containing *two* accept states. The lexer never confuses the two because of two deterministic tie-breaks applied in a fixed order:

1. **Longest match.** The lexer keeps reading while the DFA has a transition, then backs up to the last accepting state. On **whilex**, the keyword machine died after **while** but the identifier machine is still alive; the longest accepted prefix is **whilex**, whose state contains only  $p_1$ : identifier.
2. **Rule priority.** Only when the longest match is accepted by *both* branches (input exactly **while**, state  $\{q_5, p_1\}$ ) is the tie broken by the order of the rules in the specification, and the keyword rule is listed first.

So one left-to-right pass, one table lookup per character, and every input has exactly one answer.

## 2 Syntax analysis: a new grammar

The grammar:  $S \rightarrow \text{while } ( E ) S \mid \text{id} = E, \quad E \rightarrow \text{id} \mid \text{num} \mid E < E.$

### Problem: 2(a): the parser's stack at four checkpoints on `while ( i < n ) sum = i`

#### Solution

We use the top-down (predictive) picture from lecture: the stack holds the grammar symbols still to be matched, top on the left; a terminal on top is matched against the next input token and popped; a nonterminal on top is replaced by the right-hand side of a production. Tokens: **while** ( **id**(i) < **id**(n) ) **id**(sum) = **id**(i).

| checkpoint                                                                           | stack (top on the left)                   | remaining input                         |
|--------------------------------------------------------------------------------------|-------------------------------------------|-----------------------------------------|
| 0: start                                                                             | $S$                                       | <code>while ( i &lt; n ) sum = i</code> |
| expand $S \rightarrow \text{while } ( E ) S$                                         | <code>while (</code> $E$ <code>) S</code> | <code>while ( i &lt; n ) sum = i</code> |
| 1: after matching <code>while</code> and <code>(</code>                              | $E$ <code>) S</code>                      | <code>i &lt; n ) sum = i</code>         |
| expand $E \rightarrow E < E$ , then $E \rightarrow \text{id}$ , match <code>i</code> | $< E$ <code>) S</code>                    | <code>&lt; n ) sum = i</code>           |
| 2: after matching <code>&lt;</code>                                                  | $E$ <code>) S</code>                      | <code>n ) sum = i</code>                |
| expand $E \rightarrow \text{id}$ , match <code>n</code> ; match <code>)</code>       | $S$                                       | <code>sum = i</code>                    |
| 3: after matching <code>)</code>                                                     | $S$                                       | <code>sum = i</code>                    |
| expand $S \rightarrow \text{id} = E$ ; match <code>sum, =</code>                     | $E$                                       | <code>i</code>                          |
| expand $E \rightarrow \text{id}$ ; match <code>i</code>                              | (empty)                                   | (empty)                                 |
| 4: end                                                                               | (empty)                                   | (empty): accept                         |

The tree built along the way:

$$S(\text{while}, (, E(E(i), <, E(n)), ), S(\text{sum}, =, E(i))).$$

*Remark.* The production  $E \rightarrow E < E$  is left-recursive and ambiguous, so a real predictive parser would first rewrite it ( $E \rightarrow T E'$ ,  $E' \rightarrow < T E' \mid \varepsilon$ ,  $T \rightarrow \text{id} \mid \text{num}$ ). The checkpoints above show the symbol-by-symbol logic, which is what the question asks for; with the rewritten grammar the stack contents at the four checkpoints are the same up to the names of the nonterminals.

#### Problem: 2(b): top of stack after `while` and `(`; why literals never linger

##### Solution

The top is the nonterminal  $E$  (stack  $E$  `) S`). Terminals never linger because a terminal on top of the stack is immediately compared with the current input token and, on success, popped, consuming that token. The stack only ever holds *pending work*: symbols the parser has predicted but not yet matched. `while` and `(` were predicted when  $S$  was expanded and matched on the very next two steps, so they exist on the stack for exactly one step each and are gone by the first checkpoint.

#### Problem: 2(c): who checks that `i` and `n` were declared, and why the grammar cannot

##### Solution

**Semantic analysis**, using a symbol table built while walking the syntax tree: every declaration inserts a name (with its scope and type), every use looks the name up, and a missing entry is a “use of undeclared variable” error.

The grammar cannot do it because a context-free grammar only sees *structure*, not *identity*. The token `id` is one terminal symbol; the grammar has no way to say “*this* `id` must be the same string as *that* `id` that appeared earlier, arbitrarily far away.” Doing so requires remembering an

unbounded set of names and matching them at unbounded distance, which is more memory than a pushdown automaton's single stack provides (see Problem 3). A grammar with one production per possible identifier name would need infinitely many productions.

### 3 Semantic analysis: beyond context-free

**Problem: 3(a):  $\{a^n b^n c^n\}$  and declare-before-use**

#### Solution

Both are failures of the same kind: a pushdown automaton has one stack, and a stack can match *two* unbounded groups but not *three*, and can check a nested (last-in, first-out) correspondence but not a copied (first-in, first-out) one. For  $a^n b^n c^n$  the automaton can push the  $a$ 's and pop them against the  $b$ 's, but by then the count is gone and nothing remains to check the  $c$ 's; the pumping lemma makes this precise. Declare-before-use abstracts to the copy language  $\{w \# w\}$  ("the name declared here is the same name used there"): recognising it would require replaying the first  $w$  in order at an arbitrary later point, but a stack returns symbols in *reverse* order and only once. In both cases the property relates several unbounded parts of the input in a way that a single last-in-first-out memory cannot support, so no context-free grammar generates exactly the valid programs.

**Problem: 3(b): how a real compiler enforces it anyway**

#### Solution

It does not try to enforce it with the grammar. The parser accepts a superset (every syntactically well-formed program, declared or not) and produces a syntax tree; a separate **semantic-analysis pass** walks that tree with a **symbol table**, a scoped dictionary from names to declarations. Entering a block pushes a scope; a declaration inserts into the current scope; a use looks the name up through the enclosing scopes and reports an error if it is absent. This is ordinary program logic running on a Turing-complete machine, not a grammar, and it costs about one hash-table operation per identifier: linear time in practice.

**Problem: 3(c): why not a context-sensitive grammar?**

#### Solution

Theoretically possible, but the membership problem for context-sensitive grammars (equivalently, for linear-bounded automata) is **PSPACE-**

**complete**, the class named in lecture. No known algorithm decides it in polynomial time, and the general parsing algorithms for Type 1 grammars run in exponential time. A compiler that took exponential time in the length of the source, just to check declarations, would be unusable, when the symbol-table approach does the same job in linear time. There is also an engineering reason: a context-sensitive grammar encoding scoping rules would be enormous and unreadable, and every later phase (type checking, overload resolution) would have to be folded into it too. Separating a fast context-free parser from hand-written semantic checks is both faster and maintainable.

## 4 Optimisation: reading a real compiler

**Problem: 4(a): the function and both assembly outputs**

### Solution

```
int findIndex(const int *a, int n, int target) {
    for (int i = 0; i < n; i++) {
        if (a[i] == target)
            return i;
    }
    return -1;
}
```

Compiled for x86-64 (clang 17, target x86\_64-linux-gnu; gcc's output differs in register names and label style but shows the same patterns). AT&T syntax, destination last.

## -O0

```
findIndex:
    pushq    %rbp
    movq     %rsp, %rbp
    movq     %rdi, -16(%rbp)    # a
    movl     %esi, -20(%rbp)    # n
    movl     %edx, -24(%rbp)    # target
    movl     $0, -28(%rbp)      # i = 0
.LBB0_1:
    movl     -28(%rbp), %eax
    cmpl     -20(%rbp), %eax    # i < n
    ?
    jge      .LBB0_6
    movq     -16(%rbp), %rax
    movslq   -28(%rbp), %rcx
    movl     (%rax,%rcx,4), %eax # a[i]
    cmpl     -24(%rbp), %eax
    jne      .LBB0_4
    movl     -28(%rbp), %eax
    movl     %eax, -4(%rbp)      # result
    = i
    jmp      .LBB0_7
.LBB0_4:
    jmp      .LBB0_5
.LBB0_5:
    movl     -28(%rbp), %eax
    addl     $1, %eax
    movl     %eax, -28(%rbp)    # i++
    jmp      .LBB0_1
.LBB0_6:
    movl     $-1, -4(%rbp)      # result
    = -1
.LBB0_7:
    movl     -4(%rbp), %eax
    popq     %rbp
    retq
```

## -O2

```
findIndex:
    movl     $-1, %eax          #
    default result
    testl    %esi, %esi
    jle      .LBB0_5            # n <=
    0: return -1
    movl     %esi, %esi         # zero-
    extend n
    xorl     %ecx, %ecx         # i = 0
.LBB0_2:
    cmpl     %edx, (%rdi,%rcx,4) # a[i]
    == target ?
    je       .LBB0_4
    incq     %rcx                # i++
    cmpq     %rcx, %rsi
    jne      .LBB0_2
.LBB0_5:
    retq
.LBB0_4:
    movl     %ecx, %eax         # return
    i
    retq
```

## Problem: 4(b): differences that match lecture patterns

### Solution

- **Register allocation.** At -O0 every variable lives in a stack slot (a at -16(%rbp), n at -20, target at -24, i at -28, the result at -4) and is reloaded before every use. At -O2 they live in registers for the whole function: a stays in rdi, n in rsi, target in edx, i in rcx, and the result is built directly in eax.
- **No stack frame.** pushq %rbp; movq %rsp,%rbp; ...; popq %rbp disappears because nothing needs to be spilled.
- **Instruction fusion / addressing-mode use.** The -O0 sequence “load a, sign-extend i, load a[i], compare” (four instructions) becomes one: cmpl %edx, (%rdi,%rcx,4), which does the address computation, the load and the compare in a single instruction.
- **Zero idiom and loop inversion.** xorl %ecx,%ecx sets i = 0; the loop is rotated so that the exit test sits at the bottom (cmpq; jne) with one guard testl; jle before entry, halving the branches per

iteration. The -O0 code also contains a jump to a jump (.LBB0\_4: jmp .LBB0\_5) that -O2 threads away.

- **Early return as a separate exit block.** The found case jumps to .LBB0\_4, moves `i` into `eax` and returns; the not-found path already has -1 in `eax` from the very first instruction, so it returns with no extra work. (This function has no conditional move: an early return with a data-dependent index is not a pure select, so the compiler keeps a branch; the `cmovge` rewrite from lecture appears when both outcomes compute a value and one is chosen.)

#### Problem: 4(c): counting stack memory accesses

#### Solution

Instructions referencing a stack slot `-k(%rbp)`:

|                                                                | -O0 | -O2 |
|----------------------------------------------------------------|-----|-----|
| prologue: spill the three arguments, initialise <code>i</code> | 4   | 0   |
| per loop iteration (test 2, body 3, increment 2)               | 7   | 0   |
| on exit: store the result, reload it (either path)             | 2   | 0   |
| total distinct instructions touching the stack                 | 15  | 0   |

At -O2 the only memory access in the loop is the load of `a[i]` itself, which no compiler can remove because the data live in memory. Everything else that touched memory at -O0 was *the compiler's own bookkeeping*. Under the lecture's cost model (a register operand is essentially free, a cache hit costs a few cycles, a miss tens to hundreds), the -O0 loop body issues seven stack accesses plus one array access per iteration against one array access at -O2: the unoptimised loop is dominated by traffic that carries no information about the problem. Register allocation is the single most valuable optimisation precisely because it converts this traffic into zero-cost register reads.

## 5 Decidability: a reduction proof

#### Problem: 5(a): what “decidable” would mean

#### Solution

ALWAYS-HALTS-OPT would be decidable if there existed a Turing machine (a program)  $D$  that, given *any* encoding  $\langle O \rangle$  of an optimisation pass, always halts, and accepts exactly when  $O$  halts on every input program and rejects otherwise. In particular  $D$  must never loop, and it must be correct for every possible  $O$ , not just for well-behaved ones.

### Problem: 5(b): undecidability by reduction from $\text{HALT}_{TM}$

#### Solution

Suppose, for contradiction, that a decider  $D$  for ALWAYS-HALTS-OPT exists. We build a decider for  $\text{HALT}_{TM}$ , which is known not to exist.

**Construction.** Given  $\langle M, w \rangle$ , construct the program  $O_{M,w}$  that, on any input program  $P$ , does the following:

```
def O_Mw(P):  
    simulate M on w          # ignore P entirely  
    return P                 # the "identity optimisation"
```

This construction is computable:  $O_{M,w}$  is just a fixed template with  $\langle M, w \rangle$  pasted in.

**Correctness of the reduction.**  $O_{M,w}$  ignores its input, so its behaviour is the same on every  $P$ :

- If  $M$  halts on  $w$ , the simulation finishes and  $O_{M,w}$  returns  $P$ ; it halts on every input, so  $\langle O_{M,w} \rangle \in \text{ALWAYS-HALTS-OPT}$ .
- If  $M$  does not halt on  $w$ , the simulation never finishes and  $O_{M,w}$  halts on *no* input, so  $\langle O_{M,w} \rangle \notin \text{ALWAYS-HALTS-OPT}$ .

Hence  $\langle O_{M,w} \rangle \in \text{ALWAYS-HALTS-OPT} \iff \langle M, w \rangle \in \text{HALT}_{TM}$ .

**The contradiction.** The machine “on input  $\langle M, w \rangle$ : build  $O_{M,w}$ , run  $D$  on it, answer as  $D$  does” always halts (because  $D$  does) and decides  $\text{HALT}_{TM}$ . No such machine exists, so  $D$  does not exist: ALWAYS-HALTS-OPT is undecidable. (It is a valid optimisation pass in the required sense: when it halts it returns a program equivalent to its input.)

### Problem: 5(c): why optimisers use sound, conservative approximations

#### Solution

Questions of the form “does this hold on every execution” are undecidable in general, so no optimiser can answer them exactly for every program without sometimes looping forever or sometimes being wrong. Real optimisers therefore compute an approximation that is *sound* (it never claims a property that could be false, so the transformed program is always equivalent) and *conservative* (when in doubt it assumes the worst and leaves the code alone). The price is missed opportunities on some programs; the benefit is a compiler that always terminates and never miscompiles.

### Problem: 5(d): another undecidable compiler question

#### Solution

*Program equivalence:* “do these two functions compute the same output on every input?” (needed to validate that an optimisation preserved mean-

ing). Reduction: given  $\langle M, w \rangle$ , let  $f_1$  return 0 always and  $f_2$  simulate  $M$  on  $w$  and then return 0; they are equivalent iff  $M$  halts on  $w$  (otherwise  $f_2$  diverges where  $f_1$  returns). Other examples with the same shape: “is this pointer ever null at this line,” “does this loop ever execute more than  $k$  times,” “is this exception ever thrown,” “can this array index go out of bounds.” Each is decided exactly only by predicting whether some code runs, which is reachability, which is Halting.

## 6 Integrative: positioning a new phase

**Problem: 6(a): where does borrow checking sit in the hierarchy?**

### Solution

Not at Type 3 or Type 2. The check must know, at every program point, the *ownership state* of every variable in scope (owned, moved-from, borrowed mutably, borrowed immutably, and until where), and it must relate a use to a move that happened arbitrarily earlier, possibly on a different control-flow path. That is context-sensitive information about unboundedly many names tracked across unbounded distance, exactly the kind of correspondence that defeated context-free grammars in Problem 3; so, like declare-before-use and type checking, it belongs at the **Type 1** level of the diagram: a semantic check that runs after parsing on the syntax tree and control-flow graph, never expressed as a grammar. (If one insisted on the *exact* runtime property, “no execution ever uses a moved value,” that is a reachability question and lands at Type 0; see (b).)

**Problem: 6(b): is it decidable?**

### Solution

The exact semantic property is *not* decidable: whether a particular use is ever reached after a particular move depends on which branches execute, and deciding that in general is the Halting Problem (Problem 5). The borrow checker as implemented *is* decidable, because it does not answer the exact question. It answers a conservative one: “along *any* control-flow path in the graph, ignoring branch conditions, could a moved value be used?” That is a dataflow analysis over a finite lattice of ownership states that terminates in polynomial time. The cost of decidability is incompleteness: some programs that are safe at runtime are rejected (which is why Rust programmers occasionally restructure code to satisfy the checker).

**Problem: 6(c): one more phase, positioned and classified**

## Solution

**Definite assignment** (Java, C#): every local variable must be assigned on every path before it is read. (a) Type 1 level for the same reason as borrow checking: it needs per-variable state (assigned / maybe not) at every program point across arbitrary distance and across control-flow joins; a grammar cannot relate the write to the read. (b) The exact property (“on every execution that actually happens, the read is preceded by a write”) is undecidable by reachability. The implemented check is a forward dataflow analysis (“assigned on *all* incoming paths”), decidable and linear-ish in the program size, and conservative: `int x; if (c) x = 1; if (c) print(x);` is rejected even though it is safe, because the analysis does not know the two conditions coincide.

(Alternatives with the same analysis: exhaustiveness of `match/switch`, which is decidable and even Type-2-like when types are finite; unreachable-code warnings, whose exact form is Type 0 and whose implemented form is a conservative dataflow approximation.)